

## Taller Práctico - Bloque 5: SmartLibrary

### De clases aisladas a objetos que colaboran

*Interfaces · Asociación · Agregación · Composición · Herencia · Componentes UML*

#### 1. Propósito del taller

Evolucionar el diseño construido en el Bloque 4 para que los objetos de SmartLibrary dejen de verse como elementos aislados y comiencen a expresar relaciones estructurales y contratos de colaboración. El objetivo no es utilizar todos los tipos de relación posibles, sino seleccionar cada relación porque representa correctamente una regla o significado del problema.

#### 2. Actividad 1 – Diagnóstico de relaciones

| Elementos                    | ¿Qué relación existe en el problema?                               | ¿Existencia independiente?                                                                                                                                               | Decisión inicial               |
|------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------|
| <b>Estudiante – Préstamo</b> | El estudiante realiza o solicita un préstamo de ejemplares.        | Sí. Un estudiante existe sin estar en un préstamo activo, y un préstamo requiere de un estudiante pero sus ciclos de vida no dependen el uno del otro de forma absoluta. | Asociación                     |
| <b>Libro – Ejemplar</b>      | Un libro abstracto se materializa en múltiples ejemplares físicos. | Parcial. Un ejemplar depende de un libro para tener sentido conceptual, pero el libro conceptual existe aun si sus ejemplares se prestan o se agotan.                    | Agregación / Composición débil |
| <b>Préstamo – Renovación</b> | Un préstamo registra las                                           | No. Las renovaciones                                                                                                                                                     | Composición                    |

|                                |                                                                                             |                                                                                                                      |                    |
|--------------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|--------------------|
|                                | renovaciones realizadas a lo largo de su vigencia.                                          | pertenecen exclusivamente al historial interno de ese préstamo y pierden sentido si el préstamo no existe.           |                    |
| <b>Usuario – Estudiante</b>    | El estudiante es una especialización o rol que hereda de los datos generales de un usuario. | Sí. Conceptual y estructuralmente comparten identidad pero el rol define características propias (código, programa). | Herencia («es-un») |
| <b>Usuario – Bibliotecario</b> | El bibliotecario es un tipo de usuario del sistema encargado de la gestión interna.         | Sí. Al igual que el estudiante, comparte atributos base pero posee un turno y código de empleado propio.             | Herencia («es-un») |

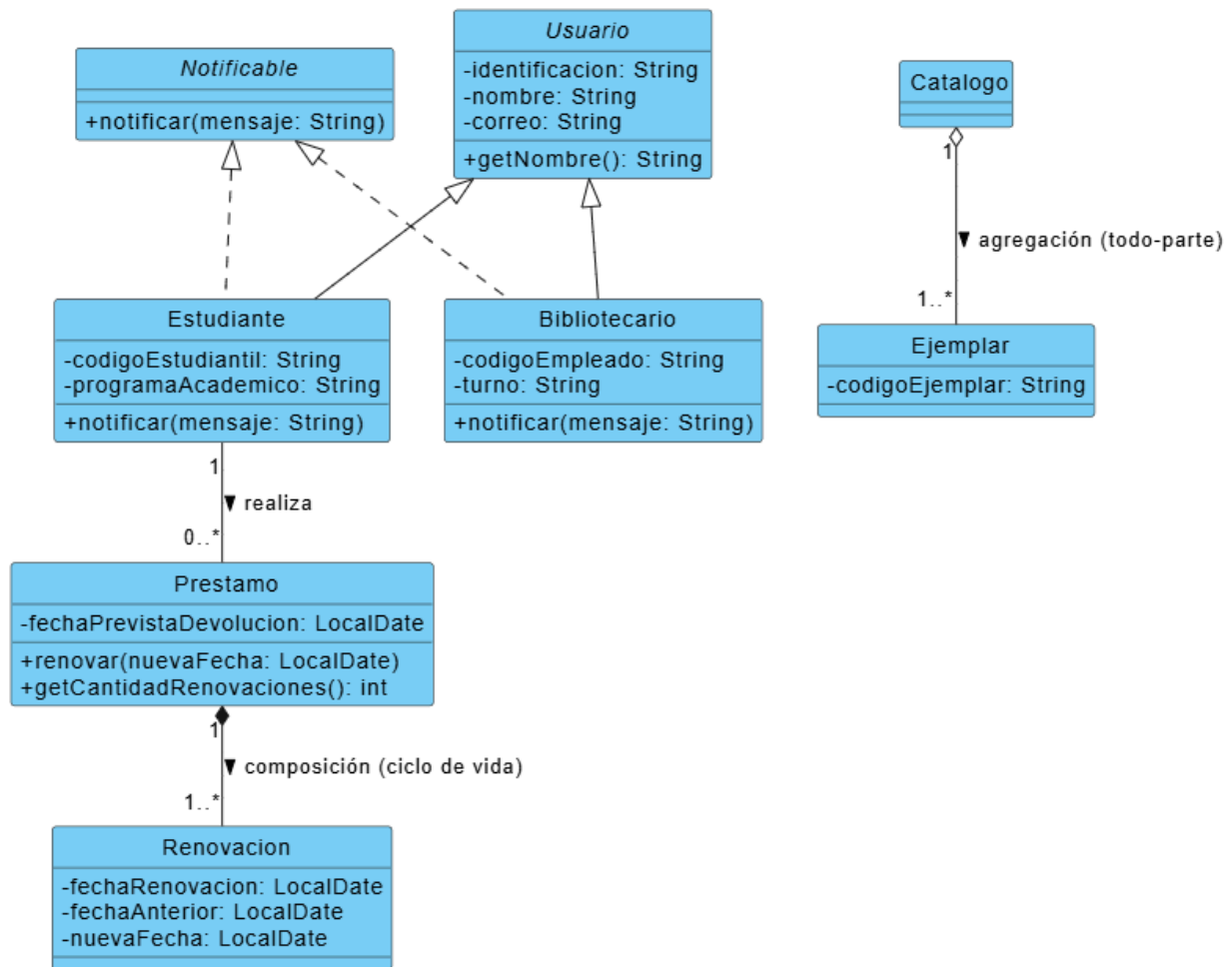
### Preguntas de discusión:

1. **¿Que dos clases de relaciones significa necesariamente que una sea parte de la otra?** No. La relación de pertenencia ("todo-parte") aplica solo para agregación o composición; muchas clases se relacionan meramente mediante colaboración o uso (asociación) sin que una contenga físicamente a la otra.
2. **¿Qué evidencia del requisito utilizaron para decidir?** En el caso de *Préstamo – Renovación*, el requisito R11 indica que "un préstamo registra las renovaciones realizadas", lo que denota dependencia estricta del ciclo de vida.
3. **¿Cuál de las relaciones anteriores sería incorrecto decidir únicamente mirando los nombres de las clases?** La relación *Libro – Ejemplar*, ya que los nombres por sí solos no indican si el ejemplar es una parte indivisible del libro (composición fuerte) o si pueden gestionarse de manera separada en el catálogo.

### 3. Actividad 2 – Asociación, agregación y composición

| Relación | Tipo seleccionado | Justificación basada en el |
|----------|-------------------|----------------------------|
|----------|-------------------|----------------------------|

|                              |                            | <b>dominio</b>                                                                                                                                                      |
|------------------------------|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Estudiante – Préstamo</b> | Asociación                 | Tienen existencia independiente; el estudiante colabora con el sistema generando préstamos a lo largo del tiempo.                                                   |
| <b>Préstamo – Ejemplar</b>   | Asociación                 | El préstamo asocia temporalmente un ejemplar físico específico a un estudiante, pero ambos mundos (inventario y transacciones) tienen ciclos independientes.        |
| <b>Libro – Ejemplar</b>      | Agregación (o Composición) | Un libro agrupa varios ejemplares físicos. Si el libro se descatalogara, los ejemplares físicos dejan de tener sentido en el sistema.                               |
| <b>Préstamo – Renovación</b> | Composición                | Las renovaciones forman parte integral del historial del préstamo; si el préstamo se elimina o archiva, las renovaciones individuales no existen de forma autónoma. |



#### 4. Actividad 3

##### 4. ¿Estudiante es realmente un tipo de usuario dentro de SmartLibrary? Justifique.

Sí, porque cumple estrictamente con la relación «es-un»: un estudiante es un tipo de usuario del sistema que comparte atributos comunes (identificación, nombre, correo) pero añade especificidades académicas.

##### 5. ¿Bibliotecario es realmente un tipo de usuario? Justifique.

Sí, comparte la misma identidad base del sistema de autenticación y gestión de perfiles, diferenciándose únicamente por sus atributos operativos de empleado (código de empleado y turno).

##### 6. ¿La existencia de identificación, nombre y correo repetidos sería suficiente para crear la superclase Usuario? ¿Por qué? No por sí sola. La reutilización de código o atributos de

texto no justifica por sí misma la herencia si no existe una jerarquía conceptual semántica real («es-un»).

**7. Propongan al menos una situación en la que crear una superclase únicamente para evitar atributos repetidos produciría un diseño conceptual débil.**

Si decidimos crear una clase Persona para unificar Autor y Estudiante solo porque ambos tienen "nombre", pero el sistema trata al autor como un metadato estático del catálogo y al estudiante como un actor interactivo con credenciales de préstamo, la jerarquía fallaría ante cambios en el comportamiento.

**5. Actividad 4 – Interfaces: expresar un contrato**

| Pregunta                                                                                        | Respuesta del equipo                                                                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>¿Qué promete una clase que implementa Notificable?</b>                                       | Promete que obligatoriamente incluirá y desarrollará la lógica del método <code>notificar(String mensaje)</code> , garantizando que cualquier emisor pueda enviarle avisos sin importar si es un estudiante, un bibliotecario u otro rol.                            |
| <b>¿La interfaz define cómo se envía el mensaje?</b>                                            | No. La interfaz solo define el contrato de comportamiento (el "qué"), dejando la implementación exacta (consola, correo, SMS) en manos de la clase que la implementa (el "cómo").                                                                                    |
| <b>¿Puede una clase heredar de Usuario e implementar Notificable al mismo tiempo? Explique.</b> | Sí. Java permite la herencia simple de clases ( <code>extends Usuario</code> ) combinada con la implementación de una o más interfaces ( <code>implements Notificable</code> ), cumpliendo tanto con la jerarquía conceptual como con el contrato de comportamiento. |

**6. Actividad 5 – Implementación representativa en Java**

A continuación se muestra el código modular necesario para respaldar el diseño del taller:

```
// 1. Interfaz Notificable
public interface Notificable {
    void notificar(String mensaje);
}
```

```
}
```

```
// 2. Clase abstracta Usuario
```

```
public abstract class Usuario {  
    private String identificacion;  
    private String nombre;  
    private String correo;  
  
    public Usuario(String identificacion, String nombre, String correo) {  
        this.identificacion = identificacion;  
        this.nombre = nombre;  
        this.correo = correo;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public String getIdentificacion() {  
        return identificacion;  
    }  
  
    public String getCorreo() {  
        return correo;  
    }  
}
```

```
// 3. Clase Estudiante
```

```
public class Estudiante extends Usuario implements Notificable {  
    private String codigoEstudiantil;  
    private String programaAcademico;  
  
    public Estudiante(String identificacion, String nombre, String correo, String codigoEstudiantil,  
String programaAcademico) {  
        super(identificacion, nombre, correo);  
        this.codigoEstudiantil = codigoEstudiantil;  
        this.programaAcademico = programaAcademico;  
    }  
}
```

```

    public String getCodigoEstudiantil() {
        return codigoEstudiantil;
    }

    @Override
    public void notificar(String mensaje) {
        System.out.println("[Notificación a Estudiante " + getNombre() + "]: " + mensaje);
    }
}

// 4. Clase Bibliotecario
public class Bibliotecario extends Usuario implements Notificable {
    private String codigoEmpleado;
    private String turno;

    public Bibliotecario(String identificacion, String nombre, String correo, String
codigoEmpleado, String turno) {
        super(identificacion, nombre, correo);
        this.codigoEmpleado = codigoEmpleado;
        this.turno = turno;
    }

    public String getCodigoEmpleado() {
        return codigoEmpleado;
    }

    @Override
    public void notificar(String mensaje) {
        System.out.println("[Notificación a Bibliotecario " + getNombre() + "]: " + mensaje);
    }
}

// 5. Clases de Soporte para Catálogo (Libro y Ejemplar)
public class Libro {
    private String isbn;
    private String titulo;

```

```

    public Libro(String isbn, String titulo) {
        this.isbn = isbn;
        this.titulo = titulo;
    }
}

public class Ejemplar {
    private String idEjemplar;
    private Libro libro; // Agregación

    public Ejemplar(String idEjemplar, Libro libro) {
        this.idEjemplar = idEjemplar;
        this.libro = libro;
    }
}

```

// 6. Clase Renovacion

```
import java.time.LocalDate;
```

```

public class Renovacion {
    private LocalDate fechaRenovacion;
    private LocalDate fechaAnterior;
    private LocalDate nuevaFecha;

    public Renovacion(LocalDate fechaRenovacion, LocalDate fechaAnterior, LocalDate
nuevaFecha) {
        this.fechaRenovacion = fechaRenovacion;
        this.fechaAnterior = fechaAnterior;
        this.nuevaFecha = nuevaFecha;
    }

    public LocalDate getNuevaFecha() {
        return nuevaFecha;
    }
}

```

// 7. Clase Prestamo con regla de negocio de renovación

```
import java.time.LocalDate;
```

```

import java.util.ArrayList;
import java.util.List;

public class Prestamo {
    private Estudiante estudiante;
    private Ejemplar ejemplar;
    private LocalDate fechaPrevistaDevolucion;
    private List<Renovacion> renovaciones = new ArrayList<>();

    public Prestamo(Estudiante estudiante, Ejemplar ejemplar, LocalDate
fechaPrevistaDevolucion) {
        this.estudiante = estudiante;
        this.ejemplar = ejemplar;
        this.fechaPrevistaDevolucion = fechaPrevistaDevolucion;
    }

    public void renovar(LocalDate nuevaFecha) {
        // Regla: Impide una renovación cuya nueva fecha sea igual o anterior a la fecha prevista
vigente.
        if (nuevaFecha == null || !nuevaFecha.isAfter(this.fechaPrevistaDevolucion)) {
            throw new IllegalArgumentException("Error: La nueva fecha de devolución debe ser
posterior a la fecha prevista actual (" + this.fechaPrevistaDevolucion + ").");
        }

        LocalDate fechaAnterior = this.fechaPrevistaDevolucion;
        this.fechaPrevistaDevolucion = nuevaFecha;

        Renovacion nuevaRenovacion = new Renovacion(LocalDate.now(), fechaAnterior,
nuevaFecha);
        this.renovaciones.add(nuevaRenovacion);
    }

    public LocalDate getFechaPrevistaDevolucion() {
        return fechaPrevistaDevolucion;
    }

    public int getCantidadRenovaciones() {
        return renovaciones.size();
    }
}

```

```
}
```

```
// 8. Clase Main de Prueba Mínima
```

```
import java.time.LocalDate;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Estudiante estudiante = new Estudiante("10023", "Ana Gómez", "ana.gomez@correo.com",  
"EST-9988", "Ingeniería de Sistemas");
```

```
        Libro libro = new Libro("978-3-16-148410-0", "Patrones de Diseño en Java");
```

```
        Ejemplar ejemplar = new Ejemplar("EJE-01", libro);
```

```
        Prestamo prestamo = new Prestamo(estudiante, ejemplar, LocalDate.of(2026, 10, 10));
```

```
        System.out.println("--- PRUEBA 1: Renovación Válida ---");
```

```
        try {
```

```
            prestamo.renovar(LocalDate.of(2026, 10, 20));
```

```
            estudiante.notificar("Su préstamo fue renovado con éxito.");
```

```
            System.out.println("Nueva fecha de devolución: " +  
prestamo.getFechaPrevistaDevolucion());
```

```
            System.out.println("Cantidad total de renovaciones: " +  
prestamo.getCantidadRenovaciones());
```

```
        } catch (Exception e) {
```

```
            System.out.println(e.getMessage());
```

```
        }
```

```
        System.out.println("\n--- PRUEBA 2: Renovación Inválida ---");
```

```
        try {
```

```
            prestamo.renovar(LocalDate.of(2026, 10, 15));
```

```
        } catch (IllegalArgumentException e) {
```

```
            System.out.println("[Excepción controlada correctamente]: " + e.getMessage());
```

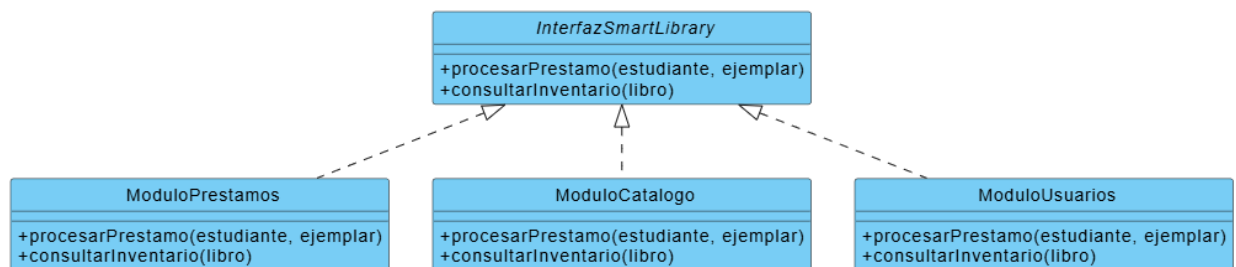
```
        }
```

```
    }
```

```
}
```

## 7. Actividad 6 – Vista funcional mediante componentes UML

| Componente                  | Clases relacionadas                | ¿De qué otro componente necesita información?<br>¿Para qué?                                                       |
|-----------------------------|------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Gestión de Usuarios</b>  | Usuario, Estudiante, Bibliotecario | Ninguno (origen de perfiles y autenticación).                                                                     |
| <b>Gestión de Catálogo</b>  | Libro, Ejemplar                    | Ninguno (gestiona inventario bibliográfico).                                                                      |
| <b>Gestión de Préstamos</b> | Prestamo, Renovacion               | Necesita de <b>Gestión de Usuarios</b> (validar estudiante) y de <b>Gestión de Catálogo</b> (verificar ejemplar). |
| <b>Gestión de Reservas</b>  | Clases de reserva del dominio      | Necesita de <b>Gestión de Usuarios</b> y <b>Gestión de Catálogo</b> .                                             |



## 8. Conclusión

Transiciones de un modelo estático a un sistema cohesivo donde los objetos colaboran mediante responsabilidades claras. El uso de asociaciones, herencia semántica e interfaces permite acoplar componentes de forma flexible, protegiendo las reglas de negocio y facilitando futuras evoluciones del software sin alterar la arquitectura global.